

# Scan a cover. Know the book.

BookWorm turns a thrift-shelf photo into a book you can trace — a full scan → label → train → ship pipeline, built by a two-person pair-programming team.

Python

YOLO

FastAPI

React  
Native

v0.2.0

Sep 2026 · roadmap & product overview

# Thriftling is fast. Figuring out what a book actually is, isn't.

## THE PROBLEM

A shelf of used books moves fast — no time to type titles or look each one up by hand.

A cover photo alone doesn't say which edition, or whether it's the book you're actually after.

Readers who buy on impulse end up with no record of what they found, or why.

## THE IDEA

Point a phone at a cover or its ISBN barcode — BookWorm reads it on the spot.

One tap saves the find to a personal library, no typing required.

A point-and-shoot companion for thrift stores, used bookshops, and shelves at home.

# One line at a time: scan first, then build everything a scanner needs to get smarter.



v0.1.0 · 2026-09-15

## Scan core

YOLO cover detection, barcode + OCR reading, HEIC support, structured logs.



M1

## Book Annotator

A local tool to hand-label real photos into ground truth.



M2

## Data pipeline

A robots.txt-respecting crawler plus Drive import to grow the photo set.



M3

## Fine-tuning

Build a YOLO dataset from labels, train, and watch it on a live dashboard.



v0.2.0 · 2026-09-17

## Mobile app

The fine-tuned model ships in your pocket — live viewfinder, tap to scan.

Every step was test-driven: a failing test first, then the fix, recorded in the changelog.

# One photo, three readings, one answer.

01 · barcode

## zxing-cpp

Reads EAN-13. Only counts a hit that starts 978/979 and passes its checksum. Decoded at full resolution.

02 · cover

## YOLO26n

Detects a book in frame using the COCO "book" class, on a 1280px copy for speed.

03 · text

## EasyOCR

Reads Spanish and English, so "Buscón" keeps its accent. Anything under 0.4 confidence is dropped.

### HANDLES THE REAL WORLD

iPhone HEIC photos load upright with correct color. ISBN-10/13 printed as text is found even when OCR splits it across two lines.

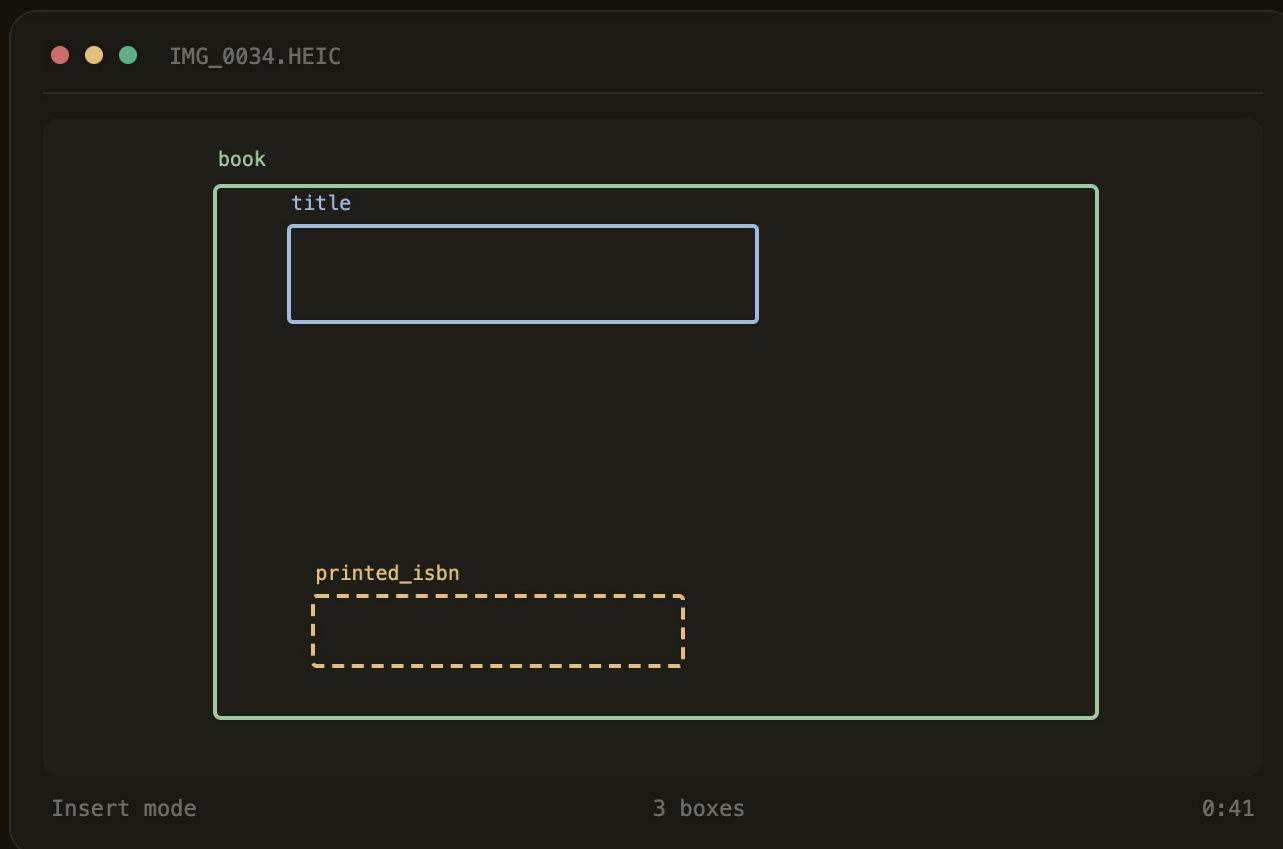
### TRACEABLE BY DESIGN

Every scan writes structured JSON logs — timestamp, level, service, call, duration — all sharing one scan\_id.

---

bookworm scan → {"kind": "isbn" | "cover" | "unknown"}

# To train a scanner, we had to become the scanner first.



## Class-first lock

Drawing is blocked until isbn, cover or unknown is picked — every label starts from the right frame.

## 7 box types

book · barcode · printed\_isbn · title · author · publisher · other\_text — nested boxes supported.

## "Detect boxes" assist

Runs the real scan pipeline and pre-draws boxes, dashed until a human confirms them.

Keyboard-first, warm dark window styled after Cursor's site — one photo labeled in under a minute.

# A crawler that grows the photo set without breaking anyone's rules.



## CRAWLING SAFELY, BY RULE NOT BY LUCK

- Every source honors robots.txt — no crawl on a disallowed path.
- Redirects are re-queued as new requests, checked against their own host.
- Response size capped at 30 MB; oversized images skipped by header.
- Only a 429 stops a run; a lone 401/403 fails just that request.
- Hosts resolving to private or loopback addresses are skipped.
- A run resumes where the last one stopped — state lives in SQLite.

---

Drive imports dedupe by file ID, so two same-named iPhone photos never silently collide.

# Labels in, a sharper detector out — and a dashboard that shows the work.

build-yolo-dataset  
7-class split, deterministic

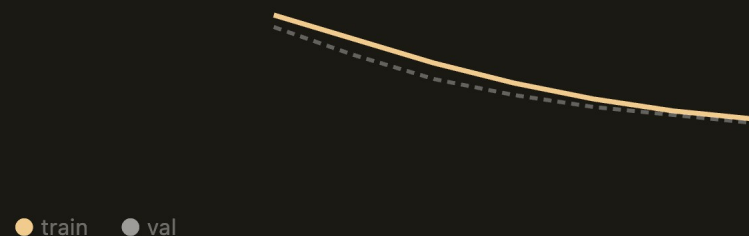
→

train-yolo  
fine-tunes yolo26n.pt

→

bookworm dashboard  
live loss + accuracy

TRAIN VS VAL LOSS



WHAT THE DASHBOARD SHOWS

Precision, recall, mAP50, mAP50-95 — each with a dashed baseline for the un-fine-tuned weights.

Parameters panel: dataset, epochs, batch size, image size, optimizer, learning rate.

A run history landing page, most recently updated first.

Unlabeled photos are skipped, not errored — labeling more later never reshuffles an existing split.

# Our own detector, trained on our own shelf.

BEFORE

**yolo26n.pt**

Base COCO weights, generic "book" class only



AFTER

**book\_detector.pt**

Fine-tuned on our 7-class dataset

**41**

hand-labeled photos

**50**

fine-tuning epochs

**7**

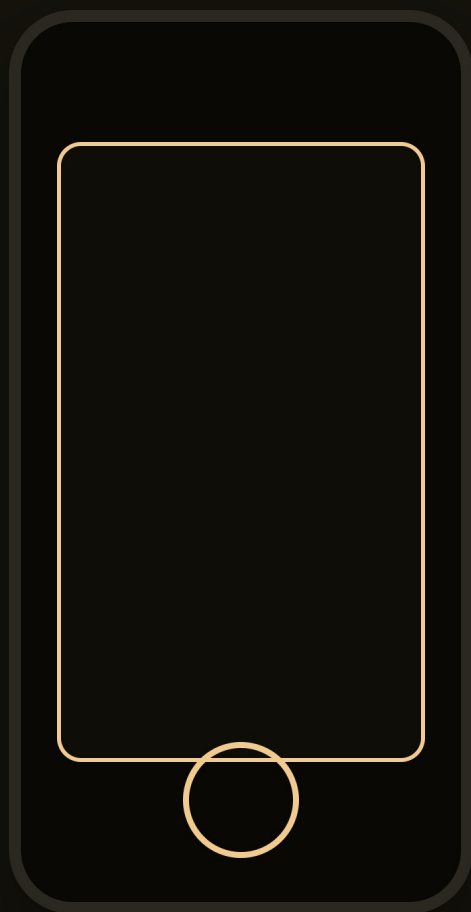
label classes

**3**

apps sharing it

scan · annotate · annotator · serve all detect books with the same fine-tuned weights now.

# The whole pipeline, held in one hand.



## Live viewfinder

A gold box tracks a framed book a few times a second over WS /api/live-detect — detector only, no OCR yet.

## Tap to scan, for real

POST /api/scan now runs the actual pipeline — fine-tuned YOLO, EasyOCR, zxing — no more hardcoded stub result.

## Biblioteca

Saved finds keep the OCR text actually read, in a warm dark "Viewfinder" look with vintage gold accents.

# The end product today: a scan that's actually right, end to end.

## V0.2.0, TODAY

Point the phone at a cover or barcode → the fine-tuned detector, EasyOCR and zxing read it → the result — real ISBN and OCR text, not a stub — saves straight to your biblioteca. Every step behind it was labeled, crawled, trained and dashboarded by tools we built ourselves.

## NEXT: IDENTIFY, NOT JUST READ

Classify title/author/publisher text automatically — today those still need a human to tag in the annotator.

## NEXT: A RESEARCH AGENT

An agent with tool calls to look up an ISBN or title and bring back edition, value and reviews.

## NEXT: MORE SHELVES, MORE DATA

Grow past 41 labeled photos with more crawler sources and real thrifting sessions.